HealthLog

Complete Project Bible

Arrhythmia Trigger Tracker

Start building after completing Sigma Web Dev tutorial #96 (Mongoose + Express).

1. Objective

People with arrhythmia are told by doctors to 'watch their lifestyle.' But they have no tool to actually see which habits affect their heart. HealthLog lets patients log their daily triggers and heart data, then surfaces plain-English insights about which combinations correlate with elevated heart rate — so they can have an informed conversation with their cardiologist.

The Gap This Fills

- Generic trackers (Apple Health, Google Fit) don't connect lifestyle habits to heart symptoms
- Cardiac apps (Cardiogram, AliveCor) require expensive hardware and don't track lifestyle triggers
- HealthLog is free, requires no hardware, and is built specifically around trigger correlation

2. Tech Stack

Layer	Technology	Why
Runtime	Node.js	JavaScript on the server
Framework	Express.js	HTTP routing and middleware
Database	MongoDB	Flexible schema for health logs
ODM	Mongoose	Schema validation and queries
Auth	JWT + bcryptjs	Stateless auth, password hashing
Frontend	HTML + CSS + Vanilla JS	No framework needed
Charts	Chart.js (CDN)	Simple, powerful visualisations
Deployment	Render + MongoDB Atlas	Free tier, easy setup

3. Folder Structure

```
healthlog/
│
├── backend/
│   ├── server.js                ← entry point, Express setup, DB connect
│   │
│   ├── models/
│   │   ├── User.js              ← user schema
│   │   ├── DailyLog.js          ← sleep, stress, water (one per day)
│   │   ├── TriggerLog.js        ← caffeine, alcohol, exercise entries
│   │   └── HeartLog.js          ← bpm + symptom entries
│   │
```

```
│   ├── routes/
│   │   ├── auth.js                ← /api/auth/register, /api/auth/login
│   │   ├── daily.js               ← /api/daily (GET, POST)
│   │   ├── trigger.js             ← /api/trigger (GET, POST, DELETE)
│   │   └── heart.js               ← /api/heart (GET, POST, DELETE)
│   │
│   ├── middleware/
│   │   └── authMiddleware.js      ← verifies JWT on protected routes
│   │
│   └── utils/
│       └── insights.js            ← all 3 correlation functions
│
├── frontend/
│   ├── index.html                 ← login / register page
│   ├── dashboard.html             ← charts + insight card + warning
│   ├── log.html                   ← all logging forms
│   ├── history.html               ← full data table, filterable
│   ├── education.html             ← static arrhythmia info cards
│   │
│   └── js/
│       ├── auth.js                ← fetch calls for login/register
│       ├── dashboard.js           ← fetch data, render Chart.js
│       ├── log.js                 ← form submissions via fetch
│       ├── history.js             ← fetch + render aggregated table
│       └── education.js           ← static, no fetch needed
│
├── .env
├── .gitignore
├── package.json
└── README.md
```

4. Database Schemas

User.js

```
{
  name:               String,   required
  email:              String,   required, unique, lowercase
  password:           String,   required  (bcrypt hashed — never stored plain)
  caffeineThreshold:  Number,   default: 400  (mg/day, user can customise)
  createdAt:          Date,     default: Date.now
}
```

DailyLog.js

One document per user per day. If user submits again same day it updates (upsert).

```
{
  user:         ObjectId  → ref: 'User',   required
  date:         String,   'YYYY-MM-DD',    required
  sleepHours:   Number,   0–24
  stressLevel:  Number,   1–5
  waterGlasses: Number,   0–20
  createdAt:    Date,     default: Date.now
}
// Compound index: { user: 1, date: 1 } unique — one doc per user per day
```

TriggerLog.js

Multiple entries per day — user logs each drink or exercise separately.

```
{
  user:      ObjectId  → ref: 'User',   required
  type:      String,   enum: ['caffeine', 'alcohol', 'exercise'],   required
```

```
  item:      String,   e.g. 'Espresso', 'Beer', 'Running'
  amount:    Number,   mg for caffeine | units for alcohol | minutes for exercise
  timestamp: Date,     default: Date.now
}
```

HeartLog.js

```
{
  user:      ObjectId  → ref: 'User',   required
  bpm:       Number,   required
  symptom:   String,   enum: [
                         'none',
                         'mild_palpitations',
                         'strong_palpitations',
                         'dizziness',
                         'chest_tightness',
                         'shortness_of_breath'
                       ]
  note:      String,   optional free text
  timestamp: Date,     default: Date.now
}
```

5. API Routes

Auth — /api/auth

```
POST /register
  body:    { name, email, password, caffeineThreshold? }
  action:  hash password → save User → sign JWT → return token
  returns: { token, user: { name, email } }

POST /login
  body:    { email, password }
  action:  find user → bcrypt.compare → sign JWT → return token
  returns: { token, user: { name, email } }
```

Daily Log — /api/daily (protected)

```
GET /
  action:  find all DailyLogs where user === req.userId
  returns: array of daily log objects

POST /
  body:    { date, sleepHours, stressLevel, waterGlasses }
  action:  findOneAndUpdate with upsert:true on { user, date }
  returns: the upserted document
```

Trigger Log — /api/trigger (protected)

```
GET /
  returns: all TriggerLogs for user, sorted by timestamp desc

POST /
  body:    { type, item, amount }
  action:  create new TriggerLog with req.userId
  returns: the created document

DELETE /:id
  action:  find by id AND user (security check) → delete
  returns: { message: 'Deleted' }
```

Heart Log — /api/heart (protected)

```
GET /       → all HeartLogs for user, sorted by timestamp desc
POST /      body: { bpm, symptom, note? }   → create HeartLog
DELETE /:id → find by id AND user → delete
```

Insights — /api/insights (protected)

```
GET /
  action:  fetch all logs → run 3 insight functions → return results
  returns: {
    strongestTrigger: { trigger, highAvgBpm, lowAvgBpm, difference, message },
    worstCombination: { highRiskAvgBpm, lowRiskAvgBpm, difference, message },
    symptomPattern:   { symptomDays, triggerName, matchCount, message }
  }
```

6. All 5 Pages — Full Detail

Page 1: index.html — Login / Register

What the user sees

- Toggle between Login and Register form
- Register: Name, Email, Password, Caffeine Threshold (optional, default 400mg)
- Login: Email, Password
- Error message area — red text for wrong password etc.

What auth.js does

```
On register submit:
  → fetch POST /api/auth/register with { name, email, password, caffeineThreshold }
  → on success: localStorage.setItem('token', token) → redirect to dashboard.html

On login submit:
  → fetch POST /api/auth/login with { email, password }
  → on success: localStorage.setItem('token', token) → redirect to dashboard.html

On every page load (runs at top of every JS file):
  → check localStorage.getItem('token')
  → if missing → redirect to index.html
```

Page 2: dashboard.html — Main Dashboard

What the user sees

- Top summary strip: Today's sleep | Today's caffeine | Today's stress | Today's water
- Warning banner (red, hidden by default): fires if caffeine total > threshold
- Chart 1 (dual-axis): caffeine bars (blue) + heart rate line (red) over 14 days
- Chart 2: sleep hours over 14 days (bar, green)
- Insight Card: 3 plain-English findings from the insight engine
- Nav links to Log, History, Education pages + Logout button

What dashboard.js does step by step

```
1. Check token → redirect if missing
2. fetch GET /api/trigger → filter to today → sum all caffeine mg
3. fetch GET /api/daily   → find today → display sleep/stress/water
4. If caffeine total > threshold → show warning banner
5. fetch GET /api/heart   → filter last 14 days → extract bpm + date
6. fetch GET /api/trigger → filter last 14 days caffeine → group by date → sum per day
7. Build Chart.js dual-axis chart (caffeine bars + bpm line)
```

```
8. Build Chart.js bar chart with sleep data
9. fetch GET /api/insights → populate insight card with 3 messages
```

Page 3: log.html — Logging Page

Section A — Morning Check-in

- Sleep hours: number input (0–24)
- Stress level: range slider (1–5) with live label
- Water glasses: number input
- Submit → green '✓ Saved' confirmation, no page reload

Section B — Log Caffeine

- Dropdown: Espresso / Americano / Latte / Green Tea / Black Tea / Energy Drink / Custom
- mg field: auto-fills from lookup table, user can override

Section C — Log Alcohol

- Dropdown: Beer / Wine / Spirits / Custom — units field auto-fills

Section D — Log Heart Rate

- BPM input, symptom dropdown, optional note textarea

Caffeine Lookup Table (in log.js)

```
const caffeineMg = {
  'Espresso':       63,
  'Americano':      77,
  'Latte':          75,
  'Green Tea':      28,
  'Black Tea':      47,
  'Energy Drink':   80,
  'Custom':         0    // user types their own
};

const alcoholUnits = {
  'Beer (pint)':    2.3,
  'Wine (glass)':   2.1,
  'Spirits (shot)': 1.0,
  'Custom':         0
};
```

Page 4: history.html — Data Table

What the user sees

- Filter bar: Last 7 / 14 / 30 days buttons
- Table: Date | Sleep | Caffeine (mg) | Alcohol (units) | Exercise (min) | Avg BPM | Worst Symptom
- Each row = one day, aggregated across all three collections
- Rows with a symptom logged are highlighted in light red

What history.js does

```
1. fetch GET /api/trigger + /api/heart + /api/daily (Promise.all — all 3 at once)
2. Build a map keyed by date:
   { sleep, stress, water, caffeineMg, alcoholUnits, exerciseMins, avgBpm, worstSymptom }
3. For each trigger log: add to right date bucket, right field
4. For each heart log: accumulate bpm values per date → average, track worst symptom
5. For each daily log: add sleep/stress/water to date bucket
6. Render into HTML table, sorted by date descending
7. Filter buttons slice the date range and re-render
```

Page 5: education.html — Info Cards (Static)

No fetch calls. Pure HTML + CSS. Six static cards:

- What is arrhythmia? — plain-English explanation
- Common triggers: caffeine, alcohol, sleep deprivation, stress, dehydration, exercise extremes
- How to read your HealthLog insights
- Safe caffeine limits (NHS / WHO guidelines — 400mg/day for healthy adults)
- Red flags — when to call your doctor: chest pain, fainting, very fast or slow pulse
- How to use this app effectively — daily 2-minute routine

7. The Insight Engine (utils/insights.js)

All three functions take the same inputs: arrays of dailyLogs, triggerLogs, heartLogs. All logic is plain JavaScript array math — no external library.

Function 1 — Strongest Single Trigger

```
THRESHOLDS (what counts as 'high' for each trigger):
  caffeine:  > 300 mg total for the day
  alcohol:   > 2 units total for the day
  sleep:     < 6 hours (inverted — LOW sleep = high risk)
  stress:    >= 4 on the 1–5 scale

ALGORITHM:
  For each trigger type:
    1. Group all logs by date → get daily totals
    2. Split dates into HIGH_RISK days and LOW_RISK days
    3. For each split, match HeartLogs on those dates → avg bpm
    4. Compute difference: highRiskAvgBpm - lowRiskAvgBpm
  Return the trigger with the largest positive difference

OUTPUT EXAMPLE:
  'Poor sleep has the strongest effect on your heart rate.
   On days with under 6 hours sleep, your average heart
   rate was 14 bpm higher than on days with sufficient sleep.'
```

Function 2 — Worst Combination

```
ALGORITHM:
  For each date that has any logs:
    Count how many triggers were in 'bad' range that day
    badCount = number of active risk factors

  Split into:
    HIGH_RISK: badCount >= 2
    LOW_RISK:  badCount <= 1

  Get avg bpm for each group from HeartLogs → compute difference

OUTPUT EXAMPLE:
  'On days with 2 or more lifestyle risk factors combined,
   your average heart rate was 18 bpm higher than on lower-risk days.'
```

Function 3 — Symptom Pattern

```
ALGORITHM:
  Find all HeartLog entries where symptom !== 'none'
  Get the dates of those entries

  For each trigger type:
    On those symptom dates, check if the trigger was elevated
    Count how many symptom days had that trigger elevated
```

```
  Return the trigger with the highest match count

OUTPUT EXAMPLE:
  'You logged palpitations on 6 days.
   On 5 of those days, you had under 6 hours of sleep.
   Poor sleep may be your strongest palpitation trigger.'
```

8. Auth Flow — Exactly How JWT Works Here

```
REGISTER:
  1. User submits { name, email, password }
  2. bcrypt.hash(password, 10) → hashedPassword
  3. new User({ name, email, password: hashedPassword }).save()
  4. jwt.sign({ userId: user._id }, process.env.JWT_SECRET, { expiresIn: '7d' })
  5. Return token to frontend
  6. Frontend: localStorage.setItem('token', token)

LOGIN:
  1. Find user by email
  2. bcrypt.compare(inputPassword, user.password)
  3. If match: jwt.sign → return token
  4. If no match: 401 'Invalid credentials'

PROTECTED ROUTES:
  Every request to /api/daily, /api/trigger, /api/heart, /api/insights:
  1. authMiddleware reads Authorization header: 'Bearer <token>'
  2. jwt.verify(token, process.env.JWT_SECRET) → decodes payload
  3. req.userId = payload.userId
  4. next() → route handler runs
  5. Route queries MongoDB with: { user: req.userId }
     → users can only ever see their own data

TOKEN EXPIRY:
  Token expires in 7 days
  If expired: jwt.verify throws → middleware returns 401
  Frontend catches 401 → clears localStorage → redirects to login
```

9. Environment Variables (.env)

```
PORT=5000
MONGO_URI=mongodb+srv://<username>:<password>@cluster0.mongodb.net/healthlog
JWT_SECRET=pick_a_long_random_string_here_minimum_32_chars
CAFFEINE_DEFAULT_THRESHOLD=400
```

Never commit .env to GitHub. Add it to .gitignore immediately.

10. package.json

```
{
  "name": "healthlog",
  "version": "1.0.0",
  "main": "backend/server.js",
  "scripts": {
    "start": "node backend/server.js",
    "dev":   "nodemon backend/server.js"
  },
  "dependencies": {
    "bcryptjs":       "^2.4.3",
```

```
    "cors":            "^2.8.5",
    "dotenv":          "^16.0.3",
    "express":         "^4.18.2",
    "jsonwebtoken":    "^9.0.0",
    "mongoose":        "^7.3.1"
  },
  "devDependencies": {
    "nodemon": "^3.0.1"
  }
}
```

11. Build

What You Build	New Concept Learned
server.js + all 4 Mongoose models + .env	Mongoose schemas, MongoDB Atlas connect
auth routes: register + login + authMiddleware	bcrypt, JWT sign + verify, middleware pattern
daily.js route (GET + POST with upsert)	findOneAndUpdate, upsert, compound index
trigger.js + heart.js routes (GET, POST, DELETE)	Full CRUD, req.userId security pattern
insights.js — all 3 functions	Array grouping by date, bucket comparison
index.html + auth.js (frontend auth)	fetch API, localStorage, redirect on 401
dashboard.html + dashboard.js + Chart.js	Async fetch, Chart.js dual-axis config
log.html + log.js (all 4 forms)	fetch POST, auto-fill lookup, setTimeout
history.html + history.js	Promise.all, data aggregation, table render
education.html + CSS polish + README	Static HTML, responsive CSS
Deploy to Render + MongoDB Atlas	Environment variables, production config
README screenshots + GitHub cleanup	Final CV-ready state

12. Deployment Steps

MongoDB Atlas

```
1. Go to mongodb.com/atlas → create free cluster
2. Create database user (username + password)
3. Whitelist 0.0.0.0/0 (allow all IPs — required for Render)
4. Get connection string → paste into .env as MONGO_URI
```

Render (Backend)

```
1. Push code to GitHub (confirm .env is in .gitignore)
2. Go to render.com → New Web Service → connect GitHub repo
3. Build command:  npm install
4. Start command:  node backend/server.js
5. Add environment variables in Render dashboard:
     PORT, MONGO_URI, JWT_SECRET
6. Deploy → live URL: https://healthlog.onrender.com
```

Frontend (GitHub Pages)

```
In all frontend fetch calls, change:
  fetch('/api/...')
To:
```

```
  fetch('https://healthlog.onrender.com/api/...')

Then push frontend folder to GitHub → Settings → Pages → deploy from /frontend
```

13. CV Description

HealthLog — Arrhythmia Trigger Tracker | Node.js · Express · MongoDB · Vanilla JS · Chart.js

Built a full-stack health application for cardiac patients to identify personal arrhythmia triggers. Users log sleep, caffeine, alcohol, stress, and exercise daily alongside heart rate and symptom episodes. A custom multi-variable insight engine written in plain JavaScript groups time-series data by day, splits entries into high and low risk buckets, and compares average heart rate across them — surfacing which lifestyle combinations correlate with elevated BPM. Features JWT authentication with bcryptjs, per-user data isolation, a real-time caffeine threshold warning, dual-axis Chart.js visualisations, and a sortable 30-day history table. Deployed on Render with MongoDB Atlas.

14. Interview Answer

"Walk me through HealthLog"

"It's a full-stack health tracker for arrhythmia patients. The backend is Express with MongoDB — three collections for daily check-ins, trigger logs like caffeine and alcohol, and heart rate entries. I used JWT for auth and bcryptjs for password hashing so passwords are never stored in plain text.

The interesting part is the insight engine — it's plain JavaScript that groups logs by day, splits days into high and low trigger buckets based on thresholds like 300mg caffeine or under 6 hours of sleep, computes the average BPM in each bucket, and returns a plain-English finding about which trigger matters most.

The frontend is Vanilla JS using the Fetch API to call my REST endpoints, and Chart.js to render a dual-axis graph showing caffeine intake and heart rate overlaid over 14 days. I deployed the backend on Render and the database on MongoDB Atlas."
